

Report CV1

Priyanshu Kumar 4-Yr B.Tech.: Mechanical Engg.,IIT(BHU)

August 2026

1 Introduction

Computer vision has become one of the most impactful fields within artificial intelligence, enabling machines to interpret and understand visual information from the world in ways that were once thought to be exclusively human capabilities. From image classification and object detection to segmentation and feature extraction, computer vision techniques are now widely applied across domains such as healthcare, autonomous vehicles, security surveillance, and industrial automation.

This project is based on computer vision and how CNN method work for image classification and segmentation related model.

This project contain CNN from scratch to pretrained model for image classification.

2 Training Setup

The model was trained on the Caltech-256 dataset using SGD with a learning rate of 0.01, a batch size of 32, and 10 epochs.

Parameter / Detail	Value
Optimizer	SGD
Learning Rate	0.01
Batch Size	32
Epochs	10
Number of Classes	257
Training Samples	21,424
Validation Samples	4,591
Testing Samples	4,592

Table 1: Training setup and dataset composition for the custom CNN on Caltech-256.

3 Task 0: Custom CNN from Scratch

3.1 Design Philosophy

Since the task explicitly discourages complexity without understanding, the architecture was designed by starting from a small, trainable baseline and scaling up depth and width only where justified. Caltech-256 contains 256 object categories with a moderate number of images per class, which means the model needs enough capacity to separate a large number of classes, but not so much that it overfits on a dataset of this size without extensive regularization. This motivated a design with progressively increasing channel depth, moderate total depth (6 convolutional layers), and strong regularization (BatchNorm + Dropout + max Pooling).

3.2 Architecture and Justification

3.2.1 Number of Layers

The network uses **six convolutional layers organized into three blocks** of two convolutions each. A single-block shallow CNN was tested first as a baseline and underfit severely on 256 classes, since two or three convolutional layers cannot build the hierarchical features (edges $\rightarrow$ textures $\rightarrow$ object parts) needed to distinguish that many categories. Beyond six convolutional layers, deeper variants gave diminishing returns while increasing overfitting risk and training time, given the absence of skip connections. Three blocks was therefore chosen as a balance between representational capacity and trainability without residual connections.

3.2.2 Kernel Size

The first two convolutional layers use a 7×7 kernel because a larger kernel captures more information from the input image, helping the model learn basic features such as edges and textures. The remaining layers use a 3×3 kernel since it requires fewer parameters, reduces computation, and is effective for learning more detailed and complex features. This combination improves feature extraction while keeping the model efficient.

3.2.3 Channel Progression (3 $\rightarrow$ 512)

Channels at each block (3 $\rightarrow$ 32, 32 $\rightarrow$ 64, 64 $\rightarrow$ 128, 128 $\rightarrow$ 256, 256 $\rightarrow$ 512, 512 $\rightarrow$ 512), following the standard convention of increasing feature capacity as spatial resolution decreases. Early layers use fewer channels (32, 64) since low-level features like edges and colors require fewer filters to represent, while deeper layers use more channels (256, 512) since high-level semantic features (needed to distinguish 256 object classes) are far more numerous and complex.

3.2.4 Batch Normalization

BatchNorm is applied **after every convolutional layer and before the activation function**. This was included from the baseline stage because, without it, the deeper blocks (512-channel layers) showed unstable and slow convergence during initial testing. BatchNorm normalizes intermediate activations, which reduces internal covariate shift, allows the use of a higher learning rate, and acts as a mild regularizer.

3.2.5 Max Pooling

2×2 max pooling with stride 2 is applied once per block (three times total), halving spatial dimensions each time. Pooling after every single convolution (rather than after every block) was tested and found to discard too much spatial information too early, hurting accuracy. Pooling once per two-convolution block preserves more spatial detail before each downsampling step while still controlling the growth of feature map size and computation.

3.2.6 Max Pooling Followed by Flattening

Instead of flattening the final 512-channel feature map directly into a large fully connected layer, **Last max pooling**. A flattening-based baseline was tested first and produced a very large first FC layer, which overfit quickly on Caltech-256's limited per-class samples.

3.2.7 Dropout

A dropout layer with $p = 0.1$ is applied between the two fully connected layers. This was added after observing a clear gap between training and validation accuracy in early experiments, indicating overfitting. Dropout at $p = 0.1$ (a standard default for FC layers) reduced this gap without noticeably slowing convergence.

3.3 Final Architecture Summary

Layer	Output Channels	Purpose
Conv1+BN1, Conv2+BN2	32, 64	Low-level feature extraction
MaxPool1	64	Downsample, reduce computation
Conv3+BN3, Conv4+BN4	128, 256	Mid-level feature extraction
MaxPool2	256	Downsample
Conv5+BN5, Conv6+BN6	512, 512	High-level semantic features
MaxPool3	512	Downsample
Global Avg Pool	512	Parameter-efficient spatial reduction
FC1 (512→256) + ReLU	256	Feature compression
Dropout ($p = 0.5$)	256	Overfitting control
FC2 (256→258)	258	Classification output

Table 2: Final architecture of the custom CNN, with 6 convolutional layers and 2 fully connected layers.

3.4 Results

Table 3 and Figure ?? report the training and validation performance of the final architecture.

Metric	Value
Training Accuracy	9.86%
Validation Accuracy	9.74%
Test Accuracy	9.73%
Training Loss	4.7201
Validation Loss	4.7692
Test Loss	4.7903

Table 3: Final performance of the custom CNN on Caltech-256.

4 Task 1: ResNet-18 CNN Architecture

4.1 Design Philosophy

Unlike the custom CNN, this architecture adopts the residual learning framework, where shortcut (skip) connections allow gradients to flow directly through the network during backpropagation. This enables the network to be considerably deeper without suffering from the degradation problem observed in plain deep CNNs. Since Caltech-256 contains 258 output classes with significant intra-class variation, a deeper architecture was required to learn more discriminative hierarchical representations while remaining computationally efficient. Therefore, a ResNet-18 inspired architecture consisting of four residual stages was implemented.

4.2 Architecture and Justification

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

ResNets Architecture taken directly from the paper — Deep Residual Learning for Image Recognition

Figure 1: ResNet architecture with residual connections showing feature map sizes at each stage.

4.2.1 Number of Layers

The network consists of an initial convolutional layer followed by four residual blocks. Each residual block contains two residual units, and each residual unit consists of two convolutional layers. In total, the architecture contains eighteen learnable layers, allowing the network to learn increasingly

4.2.2 Residual Learning

The primary motivation behind using ResNet-18 is the introduction of residual (skip) connections. Instead of learning a direct mapping, each residual block learns the residual function, which is then added to the shortcut connection. This allows gradients to propagate more effectively through the network, reducing the vanishing gradient problem and enabling stable optimization of a deeper architecture.

Whenever the input and output dimensions remain the same, identity shortcut connections are used. When the spatial resolution or number of channels changes, a 1×1 convolution with the appropriate stride is employed to match the dimensions before addition.

4.2.3 Kernel Size

The first convolutional layer uses a 7×7 kernel with stride 2 and padding 3. A larger kernel captures a wider receptive field, enabling the model to extract low-level features such as edges, corners, and textures from the input image.

All remaining convolutional layers use 3×3 kernels with padding 1. The 3×3 convolution provides an excellent balance between computational efficiency and feature extraction capability while keeping the number of model parameters manageable.

4.2.4 Channel Progression

The number of feature channels increases progressively throughout the network:

$$3 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$$

Early layers require fewer channels because they primarily learn simple visual patterns, whereas deeper layers require higher representational capacity to capture complex semantic information necessary for distinguishing among the 258 object categories.

4.2.5 Batch Normalization

Batch Normalization is applied after every convolutional layer and before the ReLU activation function. Batch normalization stabilizes the learning process, accelerates convergence, reduces internal covariate shift, and enables the use of larger learning rates. It also provides a mild regularization effect that helps improve generalization.

4.2.6 ReLU Activation

The Rectified Linear Unit (ReLU) activation function is used after every Batch Normalization layer. ReLU introduces non-linearity while avoiding saturation, allowing the network to learn complex decision boundaries efficiently and reducing the likelihood of vanishing gradients.

4.2.7 Downsampling

Spatial resolution is reduced progressively throughout the network. The initial 7×7 convolution uses stride 2, followed by a 3×3 max pooling layer with stride 2. Additional downsampling is performed at the beginning of Residual Blocks 3, 4, and 5 using stride-2 convolutions together with 1×1 shortcut convolutions. This gradually reduces computational cost while increasing the receptive field of deeper feature maps.

4.2.8 Dropout

Dropout with probability $p = 0.1$ is applied after the second convolution in each residual unit before the residual addition. A relatively small dropout rate was chosen to provide regularization while preserving the information flow through the residual connections. This helps reduce overfitting without significantly slowing convergence.

4.2.9 Global Average Pooling

Instead of flattening the final feature maps directly into a large fully connected layer, Adaptive Global Average Pooling is used to reduce each feature map to a single value. This significantly reduces the number of trainable parameters and improves generalization while maintaining the most important high-level semantic information.

4.2.10 Fully Connected Layers

After global average pooling, the resulting 512-dimensional feature vector is passed through a fully connected layer with 1000 hidden units followed by a ReLU activation. The final fully connected layer maps these features to the 258 output classes for classification.

4.3 Final Architecture Summary

Layer	Output Channels	Purpose
Conv1 (7×7) + BN + ReLU	64	Initial feature extraction
MaxPool (3×3)	64	Spatial downsampling
Residual Block 1	64	Low-level feature learning
Residual Block 2	128	Mid-level feature learning
Residual Block 3	256	High-level feature learning
Residual Block 4	512	Semantic feature extraction
Adaptive Avg Pool	512	Spatial feature reduction
FC1 ($512 \rightarrow 1000$) + ReLU	1000	Feature representation
FC2 ($1000 \rightarrow 258$)	258	Classification output

Table 4: Architecture of the implemented ResNet-18 model.

4.4 Results

Table 6 reports the training, validation, and test performance obtained using the implemented ResNet-18 architecture.

Metric	Value
Training Accuracy	29.42%
Validation Accuracy	25.75%
Test Accuracy	25.85%
Training Loss	3.1985
Validation Loss	3.4748
Test Loss	3.4672

Table 5: Performance of the implemented ResNet-18 on Caltech-256.

5 Task 3: Pretrained Model

A ResNet-18 model pretrained on ImageNet was used for this task instead of training from scratch. Since the dataset contains 257 categories with significant intra-class variation, a sufficiently deep architecture is needed, but training such a network from scratch on a moderately sized dataset risks overfitting and requires far more data and compute. Pretrained ResNet18 was chosen over other models (e.g., VGG16, AlexNet, ResNet50) because its residual connections allow better gradient flow with reduced degradation, its parameter count (11M) keeps it lightweight and less prone to overfitting compared to deeper variants, and it trains faster while still achieving strong accuracy through transfer learning. The final fully connected layer was replaced with one having 257 output units to match the number of classes, and the network was fine-tuned accordingly.

5.1 Results

Metric	Value
Training Accuracy	62.75%
Validation Accuracy	48.81%
Test Accuracy	48.71%
Training Loss	1.7555
Validation Loss	2.4088
Test Loss	2.4119

Table 6: Performance of the resnet18 model on Caltech-256.

6 Gradient-weighted Class Activation Mapping (Grad-CAM)

6.1 Overview

Gradient-weighted Class Activation Mapping (Grad-CAM) is a visualization technique used to explain the predictions of Convolutional Neural Networks (CNNs). It generates a localization heatmap that highlights the image regions that contribute the most to a specific class prediction.

Unlike saliency maps that compute gradients with respect to the input image, Grad-CAM utilizes the feature maps of the last convolutional layer, preserving spatial information while capturing high-level semantic features.

6.2 Input and Output

The Grad-CAM algorithm requires the following inputs:

- A trained convolutional neural network

- An input image
- A target convolutional layer
- A target class (optional)

Let the input image be

$$X \in R^{3 \times H \times W}$$

where

- 3 represents the RGB channels,
- H is the image height,
- W is the image width.

The network produces class scores

$$y = f(X) = [y_1, y_2, \dots, y_C]$$

where C is the total number of classes.

If no target class is specified, the predicted class is selected as

$$c = \arg \max_i y_i.$$

The output of Grad-CAM is a heatmap

$$L_{Grad-CAM}^c$$

which highlights the most discriminative image regions responsible for predicting class c .

6.3 Step 1: Forward Pass

The input image is passed through the CNN.

$$y = f(X)$$

During the forward pass, the feature maps produced by the selected convolutional layer are stored.

Suppose the selected layer generates K feature maps,

$$A = \{A^1, A^2, \dots, A^K\}$$

where

$$A^k \in R^{H' \times W'}$$

is the k^{th} activation map.

In the implementation, these feature maps are saved using a forward hook.

6.4 Step 2: Backward Pass

The score corresponding to the target class,

$$y^c,$$

is backpropagated through the network.

This computes the gradients

$$\frac{\partial y^c}{\partial A_{ij}^k}$$

which indicate how important each spatial location of every feature map is for the target class.

These gradients are stored using a backward hook.

6.5 Step 3: Compute Importance Weights

Each feature map receives an importance weight obtained by globally averaging its gradients.

The importance weight is

$$\alpha_k^c = \frac{1}{H'W'} \sum_{i=1}^{H'} \sum_{j=1}^{W'} \frac{\partial y^c}{\partial A_{ij}^k}$$

where

- H' is the feature map height,
- W' is the feature map width.

This operation is known as Global Average Pooling (GAP) over the gradients.

6.6 Step 4: Weighted Feature Maps

Each activation map is multiplied by its corresponding importance weight.

$$\alpha_k^c A^k$$

The weighted activation maps emphasize the features that contribute most strongly to the target class.

6.7 Step 5: Generate the Heatmap

The original Grad-CAM formulation computes

$$L_{Grad-CAM}^c = \text{ReLU} \left(\sum_{k=1}^K \alpha_k^c A^k \right)$$

where ReLU removes negative contributions,

$$\text{ReLU}(x) = \max(0, x).$$

In our implementation, the weighted activation maps are averaged instead of summed,

$$L^c = \text{ReLU} \left(\frac{1}{K} \sum_{k=1}^K \alpha_k^c A^k \right).$$

Since the heatmap is normalized afterwards, averaging and summation produce nearly identical visualizations.

6.8 Step 6: Heatmap Normalization

The heatmap is normalized between 0 and 1,

$$L_{norm} = \frac{L}{\max(L)}.$$

This normalized heatmap can then be resized to the original image dimensions and overlaid on the input image for visualization.

6.9 Implementation Details

The implemented Grad-CAM algorithm follows these steps:

1. Register a forward hook to save the activation maps of the target convolutional layer.
2. Register a backward hook to save the gradients during backpropagation.
3. Perform a forward pass through the CNN.
4. Select the predicted class if no target class is specified.
5. Backpropagate the target class score.
6. Compute the importance weights by global average pooling of the gradients.
7. Multiply the activation maps by the corresponding importance weights.

8. Average the weighted feature maps to generate the class activation map.
9. Apply the ReLU activation function.
10. Normalize the heatmap to the range $[0, 1]$.
11. Resize the heatmap and overlay it on the original image for visualization.

6.10 Image of Grad-CAM apply over image

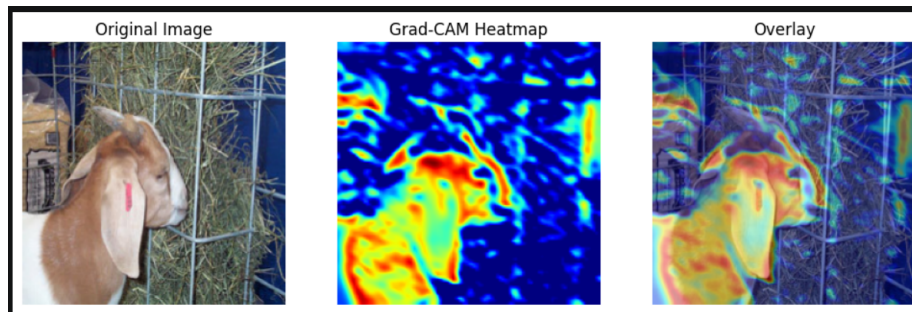


Figure 2: Grad-CAM over Task 0.

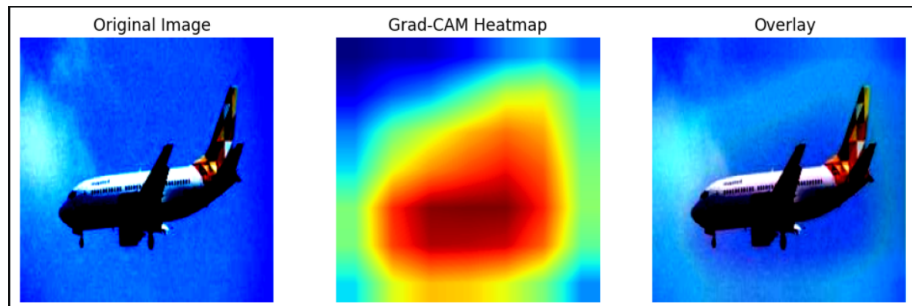


Figure 3: Grad-CAM over Task 1.